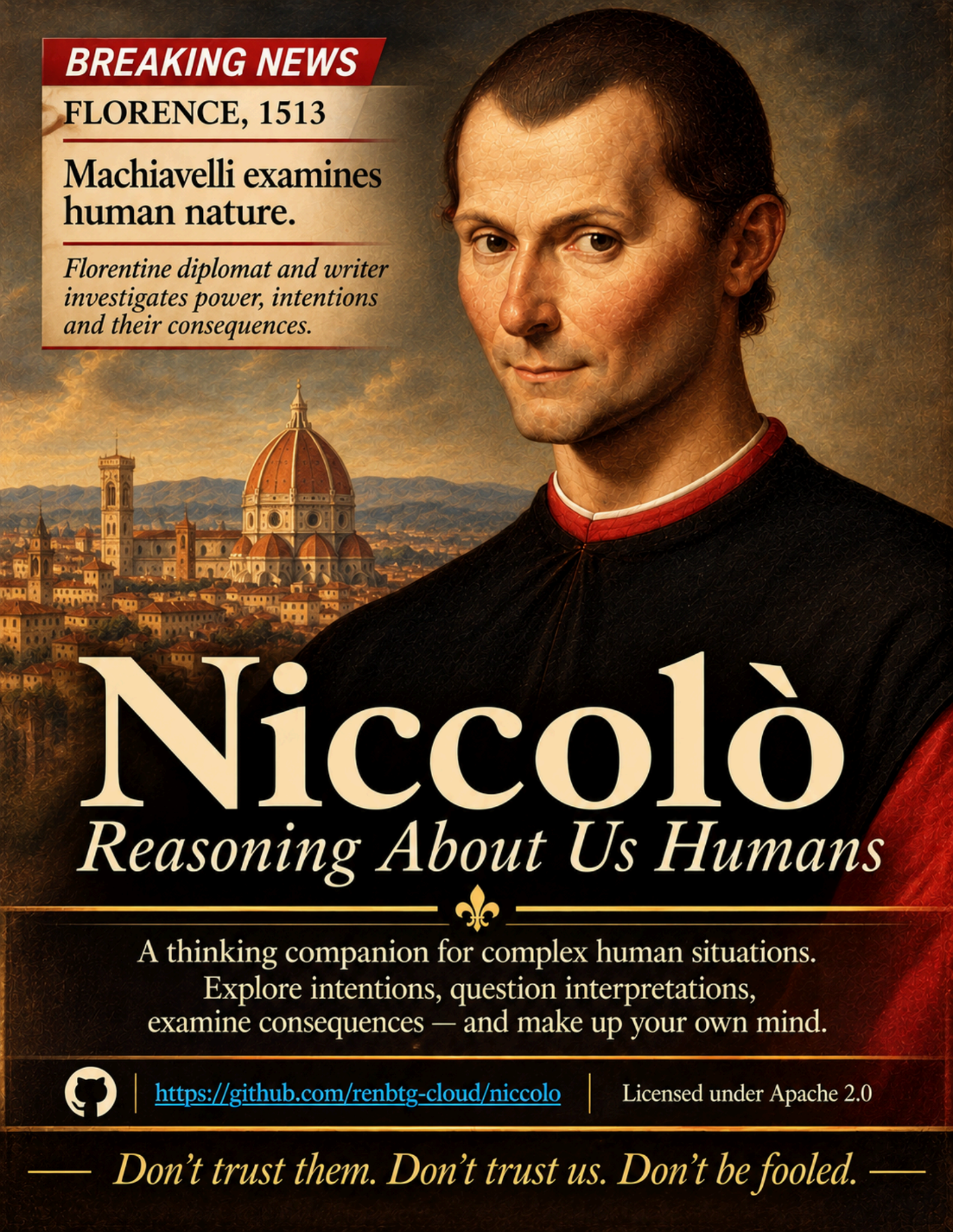

BREAKING NEWS
FLORENCE, 1513
Machiavelli examines human nature.
Florentine diplomat and writer investigates power, intentions and their consequences.
Niccolò
Reasoning About Us Humans
A thinking companion for complex human situations.
Explore intentions, question interpretations,
examine consequences — and make up your own mind.
https://github.com/renbtg-cloud/niccolo
Licensed under Apache 2.0
— Don't trust them. Don't trust us. Don't be fooled. —

Niccolò

A thinking companion for complex human situations.

Niccolò helps people examine situations where the evidence is fragmented, accounts disagree, intentions are uncertain, and a premature answer would be less useful than a better question. It brings conversations, documents, contextual contributions, competing interpretations, and possible next actions into one inspectable line of inquiry.

Its purpose is to **strengthen human judgment, not replace it**. The person defines what matters, contributes context at any moment, challenges the system's interpretations, and retains authority over consequential decisions and actions.

Niccolò is developed by Cambrian Radiation Computing. This repository contains its Rust/PostgreSQL core, local HTTP API, first-party web interface, tests, and design/validation documents.

What Niccolò does

A **Situation** is the working space for a human question or project. A Situation can combine multiple records and evolving human context without flattening uncertainty into a single authoritative story.

Capability	How it serves the person
Bring sources together	Organize email, WhatsApp exports, documents, and additional source types as they become supported. Preserve what each record says and where it came from.
Separate evidence from interpretation	Distinguish a source quotation, a person's reported belief, an inference, a hypothetical scenario, and a question worth investigating.
Reason across relationships	Compare claims, people, events, timelines, and potentially conflicting accounts without automatically declaring one of them true.
Discover useful questions	Propose a small number of promising investigative leads and explain what missing information could change the analysis.
Consider alternatives	Surface credible competing explanations, assumptions, gaps, consequences, and reasons to revise a conclusion.
Work with changing context	Let the human correct identities, retract a premise, add a source, challenge a hypothesis, or change priorities; keep the analysis history intelligible.
Plan without commandeering	Organize bounded investigation steps, note dependencies and stop conditions, and pause at decisions requiring the human.

Capability	How it serves the person
Remember the reasoning trail	Preserve provenance, frozen input revisions, feedback, and the differences between earlier and later analyses.

These are **product capabilities and directions**, not a promise that every interaction in the table is fully available in the current interface. The implementation and intended coverage are distinguished below.

A typical Situation

Imagine two people describing the same event differently in private email and a group conversation. The user can bring both records into a Situation, add a personal recollection, and ask: *What can I reasonably conclude, what remains ambiguous, and what should I ask next?*

Niccolò should show which statements have documentary support, distinguish a copied report from an independent observation, identify apparent tensions without inventing motives, and propose a few targeted ways to clarify the picture. A user can reject an interpretation as wrong, say that the evidence is insufficient, or say they do not know; those responses are not treated as interchangeable.

If new context arrives, the previous analysis remains associated with the evidence and assumptions available at the time. A new analysis can explicitly revisit it rather than rewriting history.

The same pattern applies to research, interpersonal communication, decision preparation, and other evidence-rich human situations. Niccolò is **not** a substitute for professional legal, clinical, or other regulated judgment.

Illustrative adversarial use case

For a detailed example of Niccolò reasoning about humans under conflicting incentives, uncertain provenance, distrusted operators, politically sensitive context, and a consequential decision, see **The Aurelius & Finch Bonus Review** or the **PDF edition**.

Using the product

The intended experience is organized around a Situation, rather than a single isolated prompt:

1. **Start with an intention.** Describe the situation, a concern, or a decision to explore.
2. **Add context and records.** Supply source files and explain identities or circumstances that the records cannot establish on their own.
3. **Explore.** Review attributed claims, disagreements, alternative explanations, open questions, and the system's proposed next investigative steps.
4. **Steer.** Correct, reject, question, prioritize, and add context whenever useful. A human correction creates a traceable revision rather than silently overwriting the record.
5. **Decide what happens next.** Choose whether to pursue a lead, request more information, revisit an answer, or stop. Review and approval are not automatic permission to execute external actions.

The local web interface is intentionally functional and visually restrained. Some product workflows described above require further integration; see **Product coverage**.

Architecture



- **Core:** Rust; explicit domain models for evidence attribution, provenance, Situation revisions, claims, uncertainty, review, and human control.
- **Persistence:** PostgreSQL with SQLx, workspace-scoped relationships, migrations, and row-level security boundaries.

- **Backend:** Axum HTTP API with a durable operation model. Synchronous, asynchronous, and pull-oriented observation are different ways to interact with the same underlying operation.
- **Controller:** A bounded, versioned graph of investigative steps with dependencies, hard budgets, stop conditions, and human pause/resume boundaries.
- **Frontend:** React, React Router, TanStack Query, Zustand, React Hook Form, and Zod.
- **Model layer:** provider/model interaction and evaluation are independent concerns; the core must not assume that a generated answer is evidence or that a model may authorize action.

Technical details and architectural contracts: MVP_ARCHITECTURE_V250.md, COMPETITIVE_EVALUATION.md, and the focused engineering documents in the repository.

First-time setup on one PC

If you want only the shortest user path, follow the **First-user MVP walkthrough**.

For a **new, dedicated PostgreSQL 17 database**, the MVP provides a first-run bootstrap that builds the React UI and Rust server, applies SQL migrations, creates a restricted API login, and issues a 30-day private bearer token.

Prerequisites: Node.js/npm, Rust 1.99.1, PostgreSQL 17 client tools (`psql`), and a running PostgreSQL server/database that the provided admin account owns. On Windows, use Git Bash.

Set `NICCOLO_PG_ADMIN_URL` and `NICCOLO_PG_SECURE_PASSWORD` in your shell's environment (do not include them in git commits or shared logs), then run:

```
bash scripts/bootstrap-local-mvp.sh
```

The script refuses to overwrite a previously initialized database or local credentials. It also refuses to change the password of an existing `niccolo_secure` PostgreSQL role: **roles belong to the whole PostgreSQL cluster**, not just a database. Use a dedicated cluster for an independent installation rather than letting bootstrap silently modify another installation's service credentials.

It writes private files under the ignored `.niccolo-local/` folder: `local.env` (restricted service DB URL), `identifiers.txt` (workspace and Situation identifiers), and `token.txt` (bearer token). Treat the folder as sensitive even on Windows filesystems where `chmod` may not enforce ACLs.

For a local OpenAI-compatible model runtime, configure `NICCOLO_LLM_BASE_URL` (for example `http://127.0.0.1:11434/v1`) and `NICCOLO_LLM_MODEL` before starting the API. Keyless HTTP is permitted only on loopback; remote HTTPS providers require `NICCOLO_LLM_API_KEY`. Some local models require `NICCOLO_LLM_JSON_MODE=false`.

For repeated local use, those exports may be kept in the ignored private file `.niccolo-local/model.env`; `scripts/start-local-mvp.sh` sources it automatically when present. Keep any API key in that private file, never in the prepared browser URL or repository.

Start the already-bootstrapped local MVP with one command:

```
bash scripts/start-local-mvp.sh
```

The helper prints a prepared loopback URL containing only the non-secret workspace and Situation identifiers. Open that URL, paste the bearer token from `.niccolo-local/token.txt`, and choose **Connect**. The token is deliberately never placed in the URL or printed by the start helper.

The prepared URL is also stored in `.niccolo-local/prepared-url.txt`. The first Situation is then created from the web interface by entering its focus. This is a local single-PC workflow, **not** an Internet-facing or production-hardening guide. The setup currently assumes the user has installed PostgreSQL already.

Run locally

For an MVP binary containing the **current** React interface, use `bash scripts/build-mvp.sh` after installing the prerequisites below. This command builds `frontend/dist` before compiling `niccolo-api`, whose embedded HTML/JavaScript/CSS are captured at Rust compile time. Running Cargo alone after changing React sources can serve stale UI assets.

Prerequisites

- Linux environment with Rust/Cargo compatible with this repository (the offline toolchain bundle targets Rust 1.99.1).
- PostgreSQL 17 and its client tools.
- Node.js and npm for the React frontend.
- A database used **only for development/testing**, plus restricted application credentials for API operations.
- Dependencies from the configured Cargo vendor/cache and npm lockfile, or a permitted online installation source.

The backend can be exercised independently of the frontend. Do not use live personal records for initial setup or tests.

1. Verify the Rust core

From the repository root:

```
cargo test --offline
cargo run --offline --bin niccolo-demo
```

If you have not installed the offline Cargo cache, run the equivalent commands without `--offline` after configuring access to dependencies. The exact test selection requiring PostgreSQL is controlled by the `pg-integration` feature.

2. Prepare PostgreSQL

Create a **disposable database** and separate database accounts for migration/administration and restricted runtime access. Apply SQL files from `sql/` **in ascending filename order**, including migrations added after earlier architecture checkpoints.

The API needs a dedicated non-superuser connection that passes its role and authorization checks. The database owner URL must not be supplied to the HTTP process.

Environment variables used by the repository's validation and service tooling:

Variable	Purpose
`NICCOLO_PG_ADMIN_URL`	Owner/admin connection for controlled migrations and test fixtures.
`NICCOLO_PG_SERVICE_URL`	Restricted legacy service connection used by applicable integration tests.
`NICCOLO_PG_SECURE_URL`	Dedicated restricted API connection used by the HTTP service and current validation suites.

Variable	Purpose
`NICCOLO_API_PORT`	Optional loopback API port; default `38080`.

Provisioning roles, schema migrations, and token issuance are security-sensitive setup tasks. Follow the database and API instructions in HTTP_AUTH_V007.md and the corresponding newer migration scripts; do not use an administrator URL as the runtime credential.

3. Run the API

With the secure runtime database URL configured:

```
cargo run --offline --bin niccolo-api
```

The service binds to `127.0.0.1` and defaults to port `38080`.

```
curl -i http://127.0.0.1:38080/healthz
curl -i http://127.0.0.1:38080/readyz
```

`/healthz` checks liveness; `/readyz` also evaluates selected database/runtime prerequisites. An HTTP 200 from either endpoint is not a certification of production security.

Workspace operations require a token and workspace-specific authorization. The operator CLI `niccolo-provision` is intended for trusted local provisioning, not public user registration. See HTTP_AUTH_V007.md for its command syntax and the documented API authorization boundary.

4. Run the React frontend

```
cd frontend
npm ci
npm run dev
```

To build and test it:

```
npm test
npm run build
```

The development UI and the loopback API are distinct processes. Configure their local connectivity according to your development environment; the React development server alone does not provision a PostgreSQL database or bypass API authentication.

5. Validate changes

For Rust formatting, linting, and tests:

```
cargo fmt --all -- --check
cargo clippy --offline --all-targets --features pg-integration -- -D warnings
cargo test --offline --all-targets --features pg-integration
```

Integration suites require the configured disposable PostgreSQL database, schema, and appropriate roles. The repository's `scripts/run-v*-validation.sh` scripts reproduce focused historical checkpoints. `scripts/run-v030-validation.sh`, for example, requires the three PostgreSQL URLs listed above. A successful older checkpoint is **not** a substitute for testing all subsequently changed modules and migrations.

Building on Niccolò

The backend is organized by responsibility rather than by one undifferentiated reasoning prompt:

Area	Starting points
HTTP endpoints and entrypoint	`src/api.rs`, `src/api_main.rs`
Situation revisions and storage	`src/situation.rs`, `src/situation_store.rs`
Source parsing, ingestion, and provenance	`src/source_import.rs`, `src/postgres_ingest.rs`, `src/provenance_dag.rs`
Analysis, claims, and review	`src/analysis_*.rs`, `src/review_*.rs`
Human feedback, decisions, and authorization	`src/human_*.rs`, `src/action_authorization.rs`
Durable operations and agentic orchestration	`src/bff_*.rs`, `src/agentic_*.rs`
Database schema	`sql/` sequential migrations
First-party web application	`frontend/src/`
Behavioral and integration tests	`tests/`, `frontend/src/**/*.test.js`, validation scripts

When extending the system, preserve the distinctions between *source*, *attributed statement*, *human contribution*, *machine hypothesis*, and *verified observation*. Preserve historical analysis snapshots, human-originated decisions and workspace boundaries. Tests should assert these invariants, not only happy-path HTTP results.

A useful development sequence is: identify the domain invariant; add a migration/domain contract if required; implement the service/API boundary; add adversarial and authorization tests; exercise the relevant human interaction; update documentation of the public interface.

Product coverage: implemented and planned

The two categories below are intentionally explicit. **A planned capability belongs in a product README when it explains what Niccolò is meant to become; it must not be mistaken for a working feature.**

Established engineering foundations in this repository

- Provenance-aware source and evidence representation, including multi-parent derivations and withdrawal-sensitive traversal.
- Import and attribution primitives for bounded plain-text email and WhatsApp exports; source revision and fragment tracking.
- Persistent Situations, human contributions, corrections, retractions, and immutable analysis-input snapshots.
- Claims, support relationships, alternative interpretations, uncertainties, review/feedback records, and reanalysis lineage.
- Workspace-scoped PostgreSQL persistence, documented bearer authorization boundaries, local HTTP endpoints, and integration tests.
- Durable BFF operations with multiple observation styles; structured, budgeted agentic-controller primitives.
- A first-party React interface and associated client-state/test infrastructure.

These foundations are not equivalent to a complete consumer onboarding, production-ready data security, or end-to-end autonomous analysis service.

Product direction and planned capabilities

- **Human-centered discovery:** proactive but bounded exploration of a Situation, including two or three promising investigative leads with reasons and evidence gaps.
- **Broader, reliable source coverage:** richer email and messaging ingestion, original media retention, future modalities, and cross-source identity reconciliation under human control.

- **Contextual interpretation:** consider motivations, perceptions, misunderstandings, and social dynamics as *competing hypotheses*, never asserted diagnoses or facts about people.
- **Goal-directed reasoning:** select what to investigate next according to the person's purpose, budget, stop conditions, missing evidence, and available tools.
- **Human-guided iteration:** learn from distinct kinds of feedback, preserve provenance and disagreement, and perform selective reanalysis when new information actually changes the question.
- **Model integration and empirical evolution:** evaluate candidate reasoning strategies against reproducible tests, compare model costs and quality, and improve the product without quietly delegating essential authority to a third party.
- **Usable end-to-end experience:** source upload, identity review, inspectable reasoning, informed decisions, and controlled next actions through the product interface.
- **Cohort and operational readiness:** robust authorization, privacy, retention/deletion, observability, deployment, and adversarial validation appropriate to real-world use.

The direction is not a roadmap guarantee or a claim that all these integrations are already active. Engineering checkpoint records and detailed test limitations belong in their respective technical documents, not interleaved with the product introduction.

Safety and reasoning principles

- **The human owns the decision.** Suggestions, reviews, and approvals are distinct from permission to perform an external action.
- **Attribution before assertion.** A statement about someone's beliefs remains attributed; repeated copies are not independent witnesses.
- **Uncertainty must survive.** The system should say what it does not know, present plausible alternatives, and ask questions when evidence matters.
- **History is not rewritten.** New context can justify reanalysis without retroactively changing what an earlier answer saw.
- **Security must be demonstrated.** Workspace scoping and RLS are components of a security architecture, not a substitute for a reviewed deployment boundary.

Further reading

- MVP_ARCHITECTURE_V250.md — domain, BFF, controller and frontend architecture.
- HTTP_AUTH_V007.md — local API authorization model and development provisioning.
- SITUATIONS_V016.md — Situation state and immutable revisions.
- COMPETITIVE_EVALUATION.md — benchmarking and candidate algorithm evolution.
- `sql/` and `scripts/` — executable schema and validation assets.
- Documentation guide — current product orientation and technical references.
- Engineering history — historical checkpoints, validation records, and decisions (not current product claims).